COMPARISON REPORT

Code Documentation Platform Comparison Report

Repository-Wide Code Documentation Options

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Executive Summary

This report compares the main realistic options for documenting the codebase of this repository and building a documentation workflow that can later support a user instruction such as `update code documentation`.

The repository is not a pure Python package.

It mixes

- Python source code under `scripts/`;
- extensive repository-authored Markdown documentation under `doc/`;
- YAML-driven workflows under `config/`;
- a growing family of guides that should become easier to navigate and maintain.

For that reason, the best platform is not simply the strongest API generator in isolation. The best platform is the one that can combine:

- generated Python API reference;
- hand-authored guides;
- project workflow documentation;
- maintainable automation.

The strongest overall recommendation is:

- `MkDocs + Material + mkdocstrings`

The strongest more traditional alternative is:

- `Sphinx + AutoAPI`

`Doxygen` remains viable, but it is not the most natural fit for the current repository.

Repository Fit Snapshot

The current repository profile matters for this decision.

At the time of evaluation

- Markdown files are dominant across `doc/`;
- Python files are relatively limited in number compared with the documentation tree;
- YAML configuration files are also numerous and operationally important.

This means the chosen platform must support both:

- editorial documentation;
- generated code documentation.

If the repository were only a Python package, a pure API-doc tool would be sufficient. That is not the current case.

Evaluation Criteria

The platforms were judged on

- Python API documentation quality;
- integration with Markdown-heavy documentation;
- automation suitability;
- default visual quality;
- maintainability for this repository shape;
- risk from runtime imports;
- output formats and downstream flexibility.

Decision Matrix

Platform	Python API Docs	Markdown Integration	Automation	Visual Quality	Import Risk	Repository Fit
MkDocs + Material + mkdocstrings	Strong	Excellent	Excellent	Excellent	Low to Moderate	Excellent
Sphinx + autodoc	Excellent	Moderate	Strong	Good	High	Good
Sphinx + AutoAPI	Excellent	Moderate	Strong	Good	Moderate	Good
Doxygen	Good	Moderate	Strong	Moderate	Low	Moderate
pdoc	Good	Weak	Excellent	Good	Moderate	Low to Moderate
pydoctor	Good	Weak to Moderate	Strong	Moderate	Low	Moderate

Output And Format Support

Platform	Main Outputs	Main Source Formats	Notes
MkDocs stack	HTML site	Markdown, Python via handlers	Best as a docs portal
Sphinx stack	HTML, LaTeX/PDF-oriented outputs, EPUB	reStructuredText, Markdown via plugins, Python	Strongest Python-doc ecosystem
Doxygen	HTML, PDF via LaTeX, RTF, XML	Code comments, Markdown	Multi-language heritage
pdoc	HTML	Python docstrings	Lightweight API-first option
pydoctor	HTML	Python source and docstrings	Static-analysis-oriented API docs

Repository-Grounded Example Slice

To make the comparison concrete, this evaluation uses a small real subset of the repository:

- Python module:
  - `scripts/models/feedforward_network.py`
- guide page:
  - `doc/guide/project_usage_guide.md`

  Why this slice is useful:

- `scripts/models/feedforward_network.py` is small, readable, and representative of the current Python style;

- `doc/guide/project_usage_guide.md` is long, structured, and representative of the guide-heavy nature of the repository.

The chosen platform should ideally be able to present:

- a clean API page for the `FeedForwardNetwork` class and `get_activation_module`;
- a guide/navigation page that links operational workflows and code references.

Example Source Slice

The current `scripts/models/feedforward_network.py` module includes

- `get_activation_module(activation_name: str) -> nn.Module`
- `FeedForwardNetwork(nn.Module)`
- a documented constructor with explicit runtime checks
- a `forward()` method

This is a good minimal example because it is:

- importable;
- type-annotated in parts;
- simple enough to understand;
- representative of model-code structure in the repository.

MkDocs + Material + mkdocstrings

How It Would Be Used Here

`MkDocs` would become the top-level documentation site.

The guides already authored in `doc/` would map naturally into the site navigation.

The Python module example would be exposed through an API reference page generated from source.

Example Repository Use

Possible future navigation

- `Home`
- `Usage Guide`
- `Learning Guides`
- `Training Workflows`
- `API Reference`
  - `scripts.models.feedforward_network`
  - `scripts.datasets.transmission_error_dataset`
  - `scripts.training.train_feedforward_network`

  Possible API page section for the example module:

- module summary for `scripts.models.feedforward_network`
- function section for `get_activation_module`
- class section for `FeedForwardNetwork`
- constructor parameters
- method list

  Possible guide integration:

- `project_usage_guide.md` remains a first-class page in the same site
- guide pages can link directly to API pages

Example Look

For the repository slice above, the site would likely feel like:

- left navigation for guides and API sections;
- searchable API page for `FeedForwardNetwork`;
- direct link from a training guide to the model class page;
- clean modern HTML presentation.

Example Input Pattern

Typical page pattern

- one Markdown page for narrative documentation
- one embedded API-reference directive for the Python module

Pros For This Repo

- best for mixed guide plus API documentation;
- easiest path to a future `update code documentation` command;
- strongest visual quality by default.

Cons For This Repo

- still requires docstring discipline in source;
- less library-manual feeling than a strong Sphinx API tree.

Official Links

- `MkDocs` : <https://www.mkdocs.org/>
- `Material for MkDocs` : <https://squidfunk.github.io/mkdocs-material/>
- `mkdocstrings` : <https://mkdocstrings.github.io/>

Sphinx + autodoc + autosummary + apidoc

How It Would Be Used Here

`Sphinx` would become the documentation engine.

The `feedforward_network.py` module would be documented through imported-module introspection.

The guide content could be integrated, but the repository would need a more formal documentation tree.

Example Repository Use

Possible future structure

- `index`
- `user_guide`
- `training_workflows`
- `api`
  - `scripts.models.feedforward_network`
  - `scripts.datasets.transmission_error_dataset`

    Expected API page for the example module:
- module summary;
- class signature;
- method signatures;
- docstrings rendered in API sections;
- autosummary tables for package pages.

Example Look

For the repository slice above, the result would likely feel like:

- strong module/class/function hierarchy;
- more formal API reference pages;
- less natural visual integration with the current Markdown-heavy guide set unless extra effort is invested.

Important Operational Detail

The official Sphinx documentation warns that `autodoc` imports the modules being documented.

That means import side effects must be controlled carefully.

Pros For This Repo

- strong Python-doc ecosystem;
- very solid API-reference structure;
- good long-term choice if API docs become the dominant deliverable.

Cons For This Repo

- heavier maintenance burden;
- weaker natural fit for the current documentation tree;
- import side-effect risk.

Official Links

- `autodoc` : <https://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>
- `autosummary` : <https://www.sphinx-doc.org/en/master/usage/extensions/autosummary.html>
- `apidoc` : <https://www.sphinx-doc.org/en/master/usage/extensions/apidoc.html>

Sphinx + AutoAPI

How It Would Be Used Here

This option keeps the Sphinx ecosystem, but shifts more of the API-tree generation to `AutoAPI`.

That would reduce manual maintenance compared with a heavily hand-curated `autodoc` tree.

Example Repository Use

For the example module, `AutoAPI` would generate a reference page automatically from the source tree and place it into the Sphinx site structure.

The guide content would still live alongside that generated API tree.

Example Look

The site would still feel like Sphinx, but with a more automatically generated API section.

For the repository slice

- `FeedForwardNetwork` would likely appear inside an auto-generated module tree;
- guide pages would still require explicit structure and linking work.

Pros For This Repo

- better automation than plain `autodoc` trees;
- still technically strong;
- good if you want Sphinx conventions with less manual API scaffolding.

Cons For This Repo

- still inherits the general complexity of Sphinx;
- still not as naturally aligned with a Markdown-first docs portal.

Official Links

- `AutoAPI` : <https://sphinx-autoapi.readthedocs.io/en/latest/reference/directives.html>

Doxygen

How It Would Be Used Here

`Doxygen` would parse the Python module comments and build a code-reference site.

It can also ingest Markdown files, so the repository guides could be included, but the overall documentation model would still feel API-led rather than guide-led.

Example Repository Use

For the example module

- `get_activation_module` would appear as a documented function entry;
- `FeedForwardNetwork` would appear as a class entry with methods and members;
- links among functions and classes would be generated automatically.

  For the guide side:
- `project_usage_guide.md` could be included as a Markdown page;
- however, the user experience would still feel closer to an API manual than to a documentation portal.

Example Look

For the repository slice, the result would likely feel like:

- a classic technical reference tree;
- cross-linked source entities;
- more utilitarian than modern guide portals.

Pros For This Repo

- mature and robust;
- good multi-language future path;
- scriptable and predictable.

Cons For This Repo

- less natural for the repository's current documentation culture;
- weaker fit for guide-centered navigation;
- not the strongest Python-first experience.

Official Links

- main site: <https://www.doxygen.nl/>
- Markdown support: <https://www.doxygen.nl/manual/markdown.html>
- output examples: <https://www.doxygen.nl/results.html>

pdoc

How It Would Be Used Here

`pdoc` would be a quick way to generate an API page for `feedforward_network.py`.

It would work well for module-level HTML reference, but it would not naturally become the central documentation portal for the whole repository.

Example Repository Use

For the example module

- one clean API page for the module;
- class and function signatures rendered directly;
- simple local browsing.

For the guide side:

- `project_usage_guide.md` would not become a natural first-class part of the same system without extra parallel machinery.

Example Look

For the repository slice

- the API page would be clear and readable;
- the broader docs experience would still feel fragmented.

Pros For This Repo

- very fast proof of concept;
- useful if you want to inspect one subset of the Python code immediately.

Cons For This Repo

- too limited as the main platform;
- weak fit for integrated guides plus workflows plus API reference.

Official Links

- `pdoc`: <https://pdoc.dev/docs/pdoc.html>
- example output: <https://pdoc.dev/docs/math/math_demo.html>

pydoctor

How It Would Be Used Here

`pydoctor` would generate Python API docs through static analysis rather than relying on the normal import path used by `autodoc`.

That makes it attractive for script-heavy repositories where import side effects may become annoying.

Example Repository Use

For the example module

- `FeedForwardNetwork` and `get_activation_module` would appear in an API page;
- type hints and docstrings would be included;
- the output would focus on reference documentation.

For the guide side:

- `project_usage_guide.md` would not be as naturally integrated into the same top-level system as with MkDocs.

Example Look

For the repository slice

- good API reference extraction;
- weaker integrated docs portal story.

Pros For This Repo

- attractive if import side effects must be avoided;
- solid Python API extraction.

Cons For This Repo

- smaller ecosystem;
- weaker guide-portal integration.

Official Links

- `pydoctor`: <https://pydoctor.readthedocs.io/>
- quick start: <https://pydoctor.readthedocs.io/en/latest/quickstart.html>

Comparative Example Summary

Platform	Example Module Result	Example Guide Result	Integration Quality For This Repo
MkDocs stack	Clean API page in one site	Native first-class page	Excellent
Sphinx + autodoc	Strong API reference page	Possible but heavier	Good
Sphinx + AutoAPI	Strong generated API tree	Possible but heavier	Good
Doxygen	Usable code-reference page	Possible but less natural	Moderate
pdoc	Clean quick API page	Weak integration	Low to Moderate
pydoctor	Good static-analysis API page	Weak integration	Moderate

Best First Implementation Choice

- `MkDocs + Material + mkdocstrings`

  Why:
- best match for a repository already dominated by Markdown;
- strongest single-site story for guides plus API docs;
- best user-facing output quality with the lowest friction;
- easiest path to a command like `update code documentation`.

Best Traditional Alternative

- `Sphinx + AutoAPI`

  Why:
- technically strong Python-doc stack;
- more automated than a fully manual Sphinx API tree;
- good fallback if the project later prioritizes formal API reference more heavily than narrative guides.

Option To Reject Unless You Specifically Want It

- `Doxygen` as the primary platform

  Reason:
- it is viable, but it fights the current documentation shape more than it helps it.

Automation Outlook

The future command

- `update code documentation`

  is realistic if the repository later standardizes:
- docstrings in Python modules;
- one documentation-site configuration;
- one deterministic build command;
- one validation step.

  The smoothest automation path is to let Codex do all of the following in one workflow:
- refresh API-reference pages;

- update guide navigation when needed;
- rebuild the site;
- report any build warnings or missing doc targets.

That is easiest with a `MkDocs` -based setup.

Final Decision Guidance

If the top priority is

- one coherent documentation portal for the whole repository

  choose:
- `MkDocs + Material + mkdocstrings`

  If the top priority is:
- a more formal Python API manual with stronger ecosystem convention

  choose:
- `Sphinx + AutoAPI`

  If the top priority is:
- using the only tool name you already know, even if the fit is weaker

  choose:
- `Doxygen`

  but do so knowing it is not the most repository-aligned path.

Source Links

- `MkDocs`

  <https://www.mkdocs.org/>
- `Material for MkDocs`

  <https://squidfunk.github.io/mkdocs-material/>
- `mkdocstrings`

  <https://mkdocstrings.github.io/>
- `Sphinx autodoc`

  <https://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>
- `Sphinx autosummary`

  <https://www.sphinx-doc.org/en/master/usage/extensions/autosummary.html>
- `Sphinx apidoc`

  <https://www.sphinx-doc.org/en/master/usage/extensions/apidoc.html>
- `Sphinx automatic generation tutorial`

  <https://www.sphinx-doc.org/en/master/tutorial/automatic-doc-generation.html>
- `AutoAPI`

  <https://sphinx-autoapi.readthedocs.io/en/latest/reference/directives.html>
- `Doxygen`

  <https://www.doxygen.nl/>
- `Doxygen Markdown`

  <https://www.doxygen.nl/manual/markdown.html>

- `Doxygen output examples` <https://www.doxygen.nl/results.html>
- `pdoc` <https://pdoc.dev/docs/pdoc.html>
- `pdoc example output` <https://pdoc.dev/docs/math/math_demo.html>
- `pydoctor` <https://pydoctor.readthedocs.io/>
- `pydoctor quick start` <https://pydoctor.readthedocs.io/en/latest/quickstart.html>